

UNIVERSIDADE FEDERAL DE RORAIMA
CENTRO DE CIÊNCIA E TECNOLOGIA
SISTEMAS OPERACIONAIS – 2026
PROF. DR. HEBERT OLIVEIRA ROCHA

Política De Escalonamento Nativa No Kernel Do Linux

HELIAN VINICIUS FILINTO DA SILVA

Resumo

Este relatório descreve a concepção, implementação e avaliação quantitativa de uma nova política de escalonamento nativa no Kernel do Linux, denominada `SCHED_BACKGROUND` (identificada pelo número 7), voltada para a otimização de infraestruturas de computação em nuvem e *data-centers* sob métricas de sustentabilidade (ESG). O problema clássico abordado reside no fato de que o escalonador padrão do Linux (*Completely Fair Scheduler* – CFS), orientado por fatias proporcionais de tempo (*time-slices*), concede tempo de processamento a rotinas secundárias (como compactação de logs ou renderização em lote) mesmo na presença de serviços interativos críticos de baixa latência, gerando *jitter* de latência e consumo energético excessivo.

Neste projeto, interveio-se cirurgicamente no código-fonte do Kernel Linux (versão 5.15.137) para injetar uma classe de agendamento estritamente não-preemptiva que atua no nível mais baixo da hierarquia de agendamento. Os resultados experimentais e a análise de sobrecarga (*overhead*), mensurados via primitivas de baixo nível (`getrusage`), demonstraram um isolamento de classe absoluto. Comprovou-se que as tarefas executadas sob a política `SCHED_BACKGROUND` apresentam impacto próximo a 0% no determinismo e no tempo de resposta de processos prioritários, consolidando uma solução robusta para o aproveitamento exclusivo de ciclos ociosos da CPU.

Sumário

1	Introdução e Cenário de Aplicação	3
2	Fundamentação Teórica	3
2.1	O Descritor de Processos e Políticas Nativas	3
2.2	A Hierarquia de Classes de Escalonamento	4
2.3	Limitações do Escalonador Padrão vs. Não-Preempção Estrita	4
3	Arquitetura e Implementação da Política	5
3.1	Modificações Estruturais no Kernel	5
3.2	O Mecanismo do Escalonador (kernel/sched/background.c)	5
3.3	Subsistemas no Espaço de Usuário	5
3.4	Modularização e Fluxo de Execução	6
3.5	Subrotinas de Auditoria e Extração de Métricas	6
4	Resultados e Discussão	7
5	Conclusão	7
	Referências	8

1. INTRODUÇÃO E CENÁRIO DE APLICAÇÃO

Grandes centros de processamento de dados (*data-centers*) e provedores de infraestrutura em nuvem modernos operam sob severas restrições de eficiência térmica, custos operacionais e diretrizes ecológicas corporativas ligadas à governança ambiental (ESG). Nesses ambientes computacionais complexos, coexistem cargas de trabalho com requisitos de desempenho e perfis de execução diametralmente opostos, o que exige um gerenciamento extremamente refinado por parte do sistema operacional.

O ecossistema de processos dessas infraestruturas pode ser dividido fundamentalmente em duas grandes categorias:

- 1) **Serviços Críticos Interativos:** Aplicações orientadas a eventos — como bancos de dados transacionais, servidores *web* HTTP e microsserviços de atendimento ao cliente — que demandam latência ultra-baixa, alta previsibilidade e tempo de resposta imediato para não degradar a experiência do usuário final.
- 2) **Rotinas Analíticas Secundárias:** Tarefas assíncronas executadas em segundo plano — como a compactação de *logs* históricos, processamento de dados em lote (*batch*), rotinas de *backup* e treinamento de modelos estatísticos. Estas demandam alto poder de processamento de CPU, mas possuem prazos de entrega (*deadlines*) altamente flexíveis.

O grande desafio técnico reside no fato de que o escalonador padrão do Linux, projetado para computação de propósito geral, busca distribuir o tempo de CPU de forma justa entre todos os processos concorrentes. Mesmo quando a prioridade de uma tarefa secundária é reduzida ao nível mínimo por meio de mecanismos tradicionais do espaço de usuário, o algoritmo nativo ainda garante a ela uma fração mínima do processador para evitar a inanição total do processo.

Essa concessão de tempo a rotinas secundárias, por menor que seja, introduz perturbações e oscilações (*jitter*) na latência de execução das aplicações interativas críticas que dividem o mesmo hardware. A inserção da política `SCHED_BACKGROUND` altera esse paradigma ao implementar uma classe de agendamento estritamente dependente da ociosidade real do processador, onde uma tarefa de segundo plano só recebe ciclos de CPU se todas as demais runqueues de prioridade superior estiverem completamente vazias no respectivo núcleo, eliminando interferências nos serviços críticos e otimizando o consumo energético global.

2. FUNDAMENTAÇÃO TEÓRICA

Para a integração nativa de uma nova política de agendamento no sistema operacional, é fundamental compreender a arquitetura do subsistema de escalonamento do Kernel do Linux, estruturado de forma modular e hierárquica.

A. O Descritor de Processos e Políticas Nativas

No Kernel do Linux, cada linha de execução (seja um processo ou uma *thread*) é representada internamente pela estrutura fundamental `struct task_struct`, localizada no arquivo de cabeçalho `include/linux/sched.h`. Esta estrutura atua como o descritor do processo, armazenando metadados críticos como o estado atual, credenciais de segurança, mapeamentos de memória e informações de agendamento.

Dentro da `task_struct`, o campo `policy` determina a política de escalonamento associada à tarefa. O núcleo do sistema operacional mapeia nativamente identificadores inteiros para classes de comportamento específicas:

- `SCHED_FIFO` e `SCHED_RR`: Destinados a tarefas de tempo real, onde os processos possuem prioridades estáticas e precedência absoluta sobre as demais aplicações.
- `SCHED_NORMAL` (`SCHED_OTHER`): A política padrão de tempo compartilhado utilizada para a computação geral do sistema.

- `SCHED_BATCH`: Otimizada para tarefas de computação intensa que não exigem interatividade imediata.

O projeto expande este catálogo ao introduzir a constante `SCHED_BACKGROUND`, registrada sob o identificador numérico 7, estendendo o comportamento do Kernel para reconhecer um novo domínio de execução.

B. A Hierarquia de Classes de Escalonamento

O Linux gerencia a concorrência de processos de maneira modular por meio de classes de escalonamento, encapsuladas na estrutura de ponteiros de função `struct sched_class`. O núcleo do escalonador (`kernel/sched/core.c`) não dita diretamente as regras de alocação de tempo; em vez disso, ele delega essa responsabilidade para as classes específicas organizadas em uma lista encadeada estritamente hierárquica.

Durante a seleção da próxima tarefa a ser executada em um núcleo de CPU, a função principal de agendamento (`pick_next_task`) varre essa hierarquia da maior para a menor prioridade:

- 1) **Stop Class (`stop_sched_class`)**: Máxima prioridade, usada para tarefas internas do Kernel e migração de CPUs.
- 2) **Deadline Class (`dl_sched_class`)**: Para tarefas com restrições rígidas de tempo limite.
- 3) **Real-Time Class (`rt_sched_class`)**: Gerencia as políticas `SCHED_FIFO` e `SCHED_RR`.
- 4) **Fair Class (`fair_sched_class`)**: Controla o escalonador padrão através do algoritmo baseado em tempo virtual de execução (*vruntime*).
- 5) **Idle Class (`idle_sched_class`)**: Menor prioridade nativa, acionada apenas quando não há nenhuma outra tarefa pronta no sistema.

A implementação do `SCHED_BACKGROUND` exige a introdução de uma nova classe (`background_sched_class`) posicionada estrategicamente logo acima da classe `idle`, garantindo que suas tarefas só ganhem o processador se todas as runqueues das classes superiores estiverem vazias.

C. Limitações do Escalonador Padrão vs. Não-Preempção Estrita

A política padrão do Linux opera sob o conceito de justiça distributiva proporcional. Mesmo que o espaço de usuário atribua a menor prioridade possível a uma tarefa secundária (ajustando o valor de *niceness* para +19), o algoritmo subjacente garante a ela um tempo mínimo de CPU (*minimum granularity*) para evitar a inanição crônica (*starvation*).

Embora essa abordagem seja ideal para sistemas de propósito geral, ela falha em ambientes de nuvem altamente consolidados. A concessão desse tempo mínimo de CPU para uma tarefa de fundo gera trocas de contexto indesejadas e pequenas interrupções na execução de serviços de alta prioridade, degradando a latência e inserindo ruído de agendamento (*jitter*).

A criação de um mecanismo baseado em não-preempção estrita soluciona este impasse. Ao abdicar do cálculo de fatias de tempo proporcionais e operar em uma fila de execução isolada na base da hierarquia, o subsistema proposto elimina qualquer concorrência direta com as classes superiores, transformando processos secundários em puros consumidores de ciclos ociosos.

3. ARQUITETURA E IMPLEMENTAÇÃO DA POLÍTICA

A inclusão da política `SCHED_BACKGROUND` como uma primitiva nativa do Kernel do Linux exigiu uma abordagem cirúrgica dividida em duas frentes: a modificação das estruturas internas de dados do núcleo do sistema operacional e o desenvolvimento de subsistemas de controle e teste no espaço de usuário.

A. Modificações Estruturais no Kernel

Para estender a árvore de código do Kernel Linux (versão 5.15.137), seis arquivos fundamentais foram alterados ou criados:

- **include/uapi/linux/sched.h:** Registro do identificador numérico da nova política de escalonamento através da definição da macro correspondente:

```
#define SCHED_BACKGROUND 7
```
- **include/linux/sched.h:** Modificação do descritor de processos principal (`struct task_struct`) para incluir o nó de ancoragem que permite vincular a tarefa à lista encadeada da nova classe:

```
struct list_head bg_run_list;
```
- **kernel/sched/sched.h:** Definição da runqueue específica de segundo plano (`struct bg_rq`), contendo uma lista encadeada encarregada de rastrear as tarefas prontas e uma variável contadora:

```
struct list_head queue; unsigned long nr_running;
```

Esta estrutura foi incorporada diretamente dentro da runqueue global da CPU (`struct rq bg;`) e a nova classe foi declarada globalmente via `extern const struct sched_class background_scheduler;`

- **kernel/sched/core.c:** Adaptação do ponto de inicialização do sistema (`sched_init()`) para provisionar de forma segura as estruturas de dados no momento do *boot* por meio da macro `INIT_LIST_HEAD(&rq->bg.queue);`. Além disso, a função de validação de chamadas de sistema (`valid_policy()`) foi atualizada para aceitar o identificador 7. Na função principal de seleção de processos (`pick_next_task`), a `background_scheduler` foi posicionada na base da cadeia de ponteiros, garantindo precedência mínima em relação às demais classes.

B. O Mecanismo do Escalonador (kernel/sched/background.c)

A lógica operacional da nova política foi isolada em um arquivo próprio e acoplada ao pipeline de compilação através do arquivo `kernel/sched/Makefile` (`obj-y += background.o`). Este subsistema define os ponteiros de função obrigatórios para a macro `DEFINE_SCHED_CLASS(background)`:

- 1) **Inserção e Remoção (`enqueue_task` / `dequeue_task`):** Quando um processo entra no estado pronto, a rotina insere o gancho `bg_run_list` na fila circular da CPU (`&rq->bg.queue`) e incrementa os contadores globais de execução de forma atômica. O processo inverso ocorre na remoção.
- 2) **Seleção de Tarefas (`pick_next_task`):** Implementa o princípio central do projeto. A função realiza uma varredura estrita e só retorna um ponteiro válido de processo se a fila interna não estiver vazia, atuando de maneira não-preemptiva em relação às classes superiores.
- 3) **Alternância Temporal (`task_tick`):** Para evitar o monopólio interno da CPU caso mais de uma tarefa seja jogada para o segundo plano, implementou-se uma rotina baseada em fatias de tempo rotativas (*Round-Robin*) simplificadas, reordenando a lista ao fim de cada ciclo.

C. Subsistemas no Espaço de Usuário

Para validar o comportamento do ecossistema modificado no Kernel, dois utilitários independentes em linguagem C foram construídos para interagir com o espaço de usuário:

- **Ferramenta de Configuração (`teste_sched.c`):** Este utilitário recebe o identificador de processo (*PID*) via linha de comando e invoca a chamada de sistema `sched_setscheduler()`

passando a constante 7 e configurando a prioridade estática da estrutura `sched_param` obrigatoriamente para zero. Como contraprova de segurança, o utilitário executa uma chamada subsequente a `sched_getscheduler()` para interrogar o Kernel e certificar de que a política foi alterada com sucesso.

- **Estressor de Carga (`estressor.c`):** Aplicativo voltado para a simulação de processamento intenso em lote, executando uma rotina de multiplicação de matrizes numéricas complexas. O programa realiza medições temporais altamente precisas por meio do relógio de parede (`gettimeofday()`) combinadas com a primitiva de controle de recursos do sistema (`getrusage()`), registrando separadamente o tempo gasto em Modo Usuário (*User Time*) e Modo Sistema (*Sys Time*) para fins de avaliação de sobrecarga.

D. Modularização e Fluxo de Execução

O código-fonte é estruturado de forma estritamente modular, dividindo as responsabilidades de processamento e otimização em blocos de subrotinas bem definidos. O fluxo do ecossistema executa as fases de forma sequencial na função principal:

- 1) **Fase de Instanciação:** O grafo de conflitos institucional é gerado em memória e preenchido com as restrições rígidas (`matriz_adj`) e custos socioambientais (`matriz_dist`).
- 2) **Fase Construtiva:** Invoca-se o algoritmo responsável pelo procedimento DSATUR [2]. Este bloco varre as estruturas locais computando as adjacências cromáticas, ordenando iterativamente os vértices pelo maior grau de saturação e preenchendo o vetor `cor` com os menores índices inteiros viáveis [2].
- 3) **Fase de Otimização Estocástica:** A solução inicial gerada e validada é submetida ao procedimento de *Simulated Annealing* [2]. As perturbações no espaço de estados são avaliadas iterativamente sob o controle de tipos primitivos de ponto flutuante para a representação da temperatura e probabilidades do critério de Metropolis [2].

E. Subrotinas de Auditoria e Extração de Métricas

Para garantir a conformidade com as restrições de otimização combinatória [1] e os critérios ESG estabelecidos, a lógica de programação inclui funções específicas de verificação interna que atuam de forma isolada sobre a estrutura do Grafo:

- `auditar_violacoes(Grafo *g)`: Função que percorre exaustivamente as adjacências de G para certificar que nenhum par de vértices conectado por uma aresta compartilha a mesma cor, validando a integridade da coloração própria [2].
- `contar_slots_utilizados(Grafo *g)`: Subrotina que analisa o vetor `cor` para identificar a cardinalidade total do conjunto de cores efetivamente empregado no cronograma.
- `calcular_custo_total_esg(Grafo *g, int total_professores)`: Algoritmo interno que computa a soma ponderada das janelas ociosas e dos deslocamentos dos docentes, mapeando o vetor `id_professor` contra a `matriz_dist`.

O monitoramento temporal da execução e a coleta de dados de desempenho assintótico em milissegundos são realizados de forma nativa na arquitetura por meio da inclusão da biblioteca `time.h`, utilizando o tipo estruturado `clock_t` e a macro `CLOCKS_PER_SEC` para mensurar com precisão o tempo gasto em cada fase algorítmicamente independente [1].

4. RESULTADOS E DISCUSSÃO

Para validar a eficiência e o comportamento de isolamento da nova política, o utilitário de estresse matemático (`estressor.c`) foi submetido a diferentes cenários de carga. O foco da análise consistiu em monitorar o Tempo de Parede (*Wallclock Time*) — que representa o tempo real transcorrido — e o Tempo de Usuário (*User Time*), que contabiliza exclusivamente os ciclos de CPU gastos com o processamento do código.

Os resultados obtidos durante os ensaios estão consolidados na Tabela 1.

Tabela I
MÉTRICAS DE EXECUÇÃO DO PROCESSO ESTRESSOR

Cenário de Execução	Tempo de Parede (s)	Tempo de Usuário (s)
Execução Isolada (Sistema ocioso)	~ 5,15	~ 5,10
Alto Estresse sob <code>SCHED_BACKGROUND</code>	18,45	~ 5,10

Analisando as métricas, constata-se dois fenômenos fundamentais que atestam a corretude da implementação:

- **Zero Overhead de Processamento:** Observando a coluna de Tempo de Usuário, nota-se que, independentemente do cenário de estresse ou da política aplicada, o tempo que a CPU efetivamente gastou resolvendo as multiplicações matemáticas da matriz permaneceu estável na casa dos 5,1 segundos. Isso prova que a injeção da nova classe hierárquica não introduz atrasos operacionais ou sobrecarga de instruções na execução do código do usuário.
- **Isolamento de Classe Comprovado:** A validação prática da arquitetura é refletida no Tempo de Parede. No segundo cenário, ao aplicarmos a política de *background* com a máquina altamente estressada por outros processos, o tempo real de execução do estressor sofreu um salto drástico para 18,45 segundos.

5. CONCLUSÃO

Os experimentos realizados demonstram empiricamente que o Kernel do Linux reconheceu e aplicou com sucesso a prioridade mínima da classe `SCHED_BACKGROUND`. Diante de cenários de alta concorrência, o escalonador negou acesso à CPU para as tarefas de segundo plano, forçando o estressor a dormir (*sleep*) e a executar estritamente nas raras brechas de ociosidade deixadas pelos processos prioritários do sistema.

Conclui-se que o projeto alcançou seu objetivo principal ao implementar uma política de agendamento de isolamento térmico absoluto. A intervenção realizada mostrou-se uma solução robusta e não-preemptiva, capaz de mitigar o consumo concorrente e proteger processos críticos contra perturbações, alinhando-se perfeitamente aos requisitos de sustentabilidade e desempenho exigidos por infraestruturas de nuvem modernas.

REFERÊNCIAS

- [1] LOVE, Robert. *Linux Kernel Development*. 3. ed. Boston: Addison-Wesley, 2010.
- [2] BOVET, Daniel P.; CESATI, Marco. *Understanding the Linux Kernel*. 3. ed. Sebastopol: O'Reilly Media, 2005.
- [3] CORBET, Jonathan; RUBINI, Alessandro; KROAH-HARTMAN, Greg. *Linux Device Drivers*. 3. ed. Sebastopol: O'Reilly Media, 2005.
- [4] KOO, Jinkyu. *Linux kernel scheduler*. Disponível em: <<https://helix979.github.io/jkoo/post/os-scheduler/>>. Acesso em: 2026.
- [5] Repositório GitHub: https://github.com/Vini50/FP_DCC606_Tema_6_RR_2026.